



# PRACTICA 4 – INTRODUCCIÓN SCRIPTS LINUX

## COMANDO SEQ

El comando seq en Linux genera una secuencia de números. Es útil en scripts para generar listas, rangos, o iteraciones en bucles.

### 1. Uso básico de seq: Generar una secuencia simple

```
seq 5
```

Salida:

```
1
2
3
4
5
```

### 2. Generar una secuencia con un rango específico

```
seq 2 6
```

Salida:

```
2
3
4
5
6
```

### 3. Generar una secuencia con un paso personalizado. El formato es seq inicio paso fin.

```
seq 1 2 10
```

Salida:

```
1
3
5
7
9
```

## EJEMPLOS - COMANDO SEQ

### 1. Ejemplo 1:

```
#!/bin/bash
```

```
for i in $(seq 1 2 10); do
    echo "Número: $i"
done
```

### 2. Ejemplo 2: Hacer un ping a una IP 5 veces

```
#!/bin/bash
```

```
for i in $(seq 1 5); do
    echo "Ping número $i:"
    ping -c 1 8.8.8.8
done
```



### 3. Ejemplo 3: Formato con prefijo o sufijo

```
seq -f "Número-%02g" 1 5
```

**Salida:**

```
Número-01
Número-02
Número-03
Número-04
Número-05
```

*-f permite personalizar el formato usando el especificador %g.*

### 4. Ejemplo 4: Rellenar un array con una secuencia

```
#!/bin/bash
```

```
numeros=$(seq 1 5)
echo "Array: ${numeros[@]}"
```

## OPCIONES MÁS COMUNES DE SEQ

| Opción                   | Descripción                                  |
|--------------------------|----------------------------------------------|
| <b>-w</b>                | Agrega ceros a la izquierda.                 |
| <b>-s &lt;delim&gt;</b>  | Especifica un delimitador.                   |
| <b>-f &lt;format&gt;</b> | Formato de la salida. Usa %g para el número. |

## FUNCIONES:

Una función compila un conjunto de comandos en un grupo. Si necesitas volver a ejecutar el comando, simplemente escribe la función en lugar del conjunto completo de comandos.

**Hay varias formas de escribir funciones.**

1. La primera forma es empezar con el nombre de la función y seguirlo con paréntesis y corchetes:

```
function_name () {
    first command
    second command
}
```

También puedes escribirlo en una sola línea:

```
function_name () { first command; second command; }
```

2. El segundo método para escribir una función es utilizar la palabra reservada **function** seguida del nombre de la función. Esto elimina la necesidad de paréntesis:

```
function function_name {
    first command
    second command
}
```

Este método también tiene una versión de una sola línea:



```
function function_name { first command; second command; }
```

3. Llamar a la función

```
mi_funcion
```

4. Función con parámetros: Las funciones pueden recibir argumentos que se pasan como parámetros \$1, \$2, etc.

```
#!/bin/bash
```

```
saludar() {  
    echo "Hola, $1 $2!" # Usa $1 y $2 para los argumentos  
}
```

5. Llamar a la función con parámetros

```
saludar "Juan" "Pérez"
```

6. Función con valores de retorno: Las funciones pueden devolver valores numéricos usando el comando return. Para devolver cadenas, se usa echo y captura con sustitución de comandos.

```
#!/bin/bash
```

```
sumar() {  
    local resultado=$(( $1 + $2 )) # Cálculo interno  
    return $resultado  
}
```

```
sumar 3 7  
echo "La suma es $? (valor devuelto por return)"
```

7. Variables locales dentro de funciones: Usa local para declarar variables que solo existan dentro de la función.

```
#!/bin/bash
```

```
calcular() {  
    local num1=$1  
    local num2=$2  
    echo $(( num1 * num2 ))  
}
```

```
resultado=$(calcular 5 4)  
echo "El producto es $resultado"
```

8. Manejo de errores en funciones: Combina funciones con códigos de error y mensajes de salida.

```
#!/bin/bash
```

```
dividir() {  
    if [ $2 -eq 0 ]; then  
        echo "Error: No se puede dividir por cero"
```



```
        return 1
    fi
    echo $(( $1 / $2 ))
    return 0
}

resultado=$(dividir 10 0)
if [ $? -ne 0 ]; then
    echo "La división falló."
else
    echo "El resultado es $resultado"
fi
```

## MANEJO DE COMANDOS – ERRORES Y \$

### 1. ¿Qué hace if [ \$? -eq 0 ]?

- Verifica si el último comando tuvo éxito (\$? igual a 0).
- Es una forma común de manejar errores en scripts.

### 2. ¿Qué es \$ en general?

Acceso a variables y valores especiales.

#### Ejemplos:

- **\$variable**: Muestra el valor de una variable.
- **\$?**: Código de salida del último comando.
- **\$@**: Lista de todos los argumentos del script.



## SECUENCIA / DESARROLLO

### 1. Escribir el código del script que muestre por pantalla el siguiente menú:

- Crear cuenta de usuario nueva
- Cambiar contraseña de usuario
- Borrar cuenta de usuario
- Crear grupos de usuarios
- Salir

(a) Se le pedirá al usuario que introduzca el login de la cuenta de usuario que quiera crear, modificar o borrar. Ha de comprobarse la existencia o no del usuario a la hora de crear, modificar o borrar un usuario. Para ello debemos consultar el fichero **/etc/passwd**.

(b) De la misma manera, se le pedirá el nombre del grupo de usuario que decida crear. También debe comprobarse su existencia en el fichero correspondiente. ¿Cuál es?:

(c) El menú continuará ejecutándose hasta que el usuario elija la opción Salir.

(d) Debe crear una función para cada una de las opciones del menú

### 2. Crea el script **FichPeligro.sh** que **.busque** ,en todo el sistema de archivos, cualquier fichero que pueda ser modificado por cualquier usuario (**--- --- rwx**) y guarde la lista de ficheros con la ruta exacta en el archivo **fichs\_peligrosos.txt**. **Nota:** Para buscar archivos utilizar el comando **find**

- Debe de controlar que lo ejecuta root. Si no lo ejecuta root se mostrará el mensaje "Debe de ser root.Lo siento"
- !! Ojo solo ficheros regulares
- Mostrará mensaje indicando el fichero creado y el numero de archivos encontrados (peligrosos)

### 3. Crea el script **AcertarNum.sh** que consistirá en acertar un número menor que 100. Para ello utilizamos el comando **Random**.

***echo "Número aleatorio: \$RANDOM"***

- Se generará un número al azar menor que 100.
- Se mostrará mensaje explicativo de lo que hace el script: Introducir números para intentar adivinar un número secreto menor que 100.
- Cada vez que el usuario introduzca un número y no acierte se le dará pista: el numero secreto es mayor/menor que el número introducido.
- Al final se mostrará el no de intentos hasta conseguir acertar y el tiempo empleado en acertarlo

***#!/bin/bash***

***inicio=\$(date +%s)***

***echo "Cronómetro iniciado... espera unos segundos."***

***sleep 5***

***fin=\$(date +%s)***

***duracion=\$(( fin - inicio ))***

***echo "Duración: \$duracion segundos"***



- Sólo se aceptará que el usuario introduzca números enteros válidos. No muestra mensaje de error; simplemente seguirá solicitando que introduzca un número válido.